

PhishGuard

An Explainable Machine Learning Framework for Real-Time Phishing URL Detection Using Feature Engineering and Ensemble Learning

A Final Year Project Synopsis Submitted in Partial Fulfillment of the Requirements for the Degree of
[Your Degree, e.g., Bachelor of Technology in Computer Science]

Submitted By:

Name	Roll Number	Role
[Your Name]	[Roll No.]	Team Lead — ML & Backend Development
[Member 2 Name]	[Roll No.]	Backend Development
[Member 3 Name]	[Roll No.]	Frontend Development
[Member 4 Name]	[Roll No.]	Testing & Documentation

Under the Guidance of: [Teacher/Guide Name] [Designation]

Department of [Your Department] [College/University Name] Academic Year 2025–2026

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Literature Review / Existing Systems
- 5. Objectives
- 6. Scope of the Project
- 7. Proposed System
- 8. System Requirements
- 9. Proposed Methodology
- 10. System Architecture
- 11. Feature Engineering Details
- 12. Machine Learning Models Used
- 13. Technology Stack
- 14. Expected Outcomes
- 15. Advantages of Proposed System
- 16. Limitations
- 17. Future Scope
- 18. Project Timeline
- 19. Team Contribution
- 20. Conclusion

21. References

1. Abstract

Phishing remains one of the most persistent and damaging forms of cyberattack in the modern digital landscape. Attackers craft deceptive URLs that closely mimic legitimate websites to steal sensitive user information such as login credentials, banking details, and personal identity data. Despite widespread awareness, phishing continues to be responsible for a significant percentage of global data breaches every year, largely because traditional detection systems — such as static blacklists and rule-based filters — are reactive rather than predictive, and are easily circumvented by newly registered or disguised malicious domains.

This project, **PhishGuard**, proposes an intelligent, real-time phishing URL detection system built on machine learning principles. The system extracts over 30 lexical, host-based, and content-based features from a given URL, and feeds them into an ensemble of multiple machine learning classifiers — including Random Forest, XGBoost, Support Vector Machines, and Logistic Regression — combined through a soft-voting mechanism to maximize predictive accuracy.

What differentiates PhishGuard from conventional detection systems is its emphasis on **explainability**. Using SHAP (Shapley Additive explanations), the system does not merely classify a URL as "phishing" or "legitimate" — it explains precisely *which features* contributed to that decision and to what degree, thereby building user trust and enabling security analysts to understand the reasoning behind each prediction.

The complete system is delivered as a full-stack web application, comprising a Django REST Framework backend for feature extraction and inference, a MongoDB database for persistent storage of URLs and predictions, and a React-based frontend that allows users to check URLs in real time and view detailed, human-readable explanations of the results. The system targets a detection accuracy of over 95% with an average response time under 200 milliseconds, making it suitable for real-world, real-time deployment scenarios.

2. Introduction

The internet has become an indispensable part of daily life, enabling communication, commerce, banking, and countless other services. However, this widespread reliance on online platforms has also made users increasingly vulnerable to cyberattacks — chief among them, **phishing**.

Phishing is a form of social engineering attack in which an attacker impersonates a trustworthy entity — typically through a fraudulent website or email — to trick victims into revealing sensitive information such as usernames, passwords, credit card numbers, or other personal data. These attacks often rely on URLs that are deliberately crafted to resemble legitimate domains, using techniques such as character substitution (e.g., "g00gle.com" instead of "google.com"), subdomain manipulation, URL shortening services, or embedding suspicious keywords such as "verify," "secure," or "account" to create a false sense of urgency and legitimacy.

According to recent industry reports, phishing accounts for a substantial share of reported cybercrime incidents globally, with billions of dollars lost annually due to successful attacks. Despite significant

advancements in cybersecurity, phishing techniques continue to evolve, often outpacing the static, rule-based detection mechanisms deployed by many existing security tools.

This project seeks to address this evolving threat through the application of machine learning — specifically, a combination of **feature engineering** and **ensemble learning techniques** — to build a system that can accurately, quickly, and transparently identify phishing URLs, even those not previously seen or catalogued in existing threat databases.

3. Problem Statement

Despite the availability of numerous phishing detection mechanisms, several critical gaps remain in current approaches:

3.1 Reliance on Static Blacklists Most conventional phishing detection systems (e.g., Google Safe Browsing, PhishTank-based filters) rely on maintaining databases of known malicious URLs. While effective against previously identified threats, these systems are inherently reactive — a phishing site must first be reported and verified before it can be blocked, leaving a critical window during which new phishing campaigns remain undetected.

3.2 Lack of Explainability Even where machine learning is employed, most existing solutions function as "black boxes," providing a binary classification (phishing/legitimate) without any justification. This lack of transparency reduces user trust and makes it difficult for security analysts to audit or improve detection systems.

3.3 Poor Real-Time Performance Many advanced detection systems that analyze webpage content, DOM structure, or network behavior introduce significant latency, making them unsuitable for real-time applications such as browser extensions or API-based integrations where response time is critical.

3.4 High False Negative Rates Attackers continuously adapt their techniques to bypass detection — using URL shorteners, homograph attacks, and dynamically generated subdomains — which existing static systems often fail to catch, resulting in a high rate of missed detections (false negatives).

Problem Definition: *There is a need for an intelligent, adaptive, and explainable phishing URL detection system capable of accurately classifying URLs in real time — without relying solely on static blacklists — while providing clear, interpretable justifications for its predictions.*

4. Literature Review / Existing Systems

Several approaches have been explored in prior research and commercial tools for phishing detection:

4.1 Blacklist-Based Systems Tools such as Google Safe Browsing and PhishTank maintain centralized databases of known malicious URLs. While simple and low-latency, these systems cannot detect zero-day phishing attacks and require constant manual or crowdsourced updates.

4.2 Heuristic-Based Detection Heuristic systems apply manually defined rules (e.g., "flag URLs containing an IP address" or "flag URLs with more than 5 subdomains") to identify suspicious patterns. These systems are faster than blacklists but suffer from high false positive rates and require constant rule maintenance as attacker techniques evolve.

4.3 Machine Learning-Based Detection Recent academic research has demonstrated that machine learning classifiers — including Random Forest, Support Vector Machines, and Gradient Boosting methods — can achieve high accuracy (often exceeding 95%) when trained on well-engineered URL features. Studies comparing ensemble methods against individual classifiers have consistently shown that combining multiple models yields superior and more robust performance, particularly when using techniques such as stacking or soft-voting.

4.4 Deep Learning Approaches Some research has explored deep learning models (CNNs, RNNs, and transformer-based architectures) applied directly to raw URL strings or webpage screenshots. While these methods can achieve strong accuracy, they often require substantially larger training datasets and computational resources, and their black-box nature further compounds the explainability problem.

4.5 Research Gap While ensemble learning has been shown to outperform individual classifiers, and explainability techniques such as SHAP have gained traction in other ML domains, few existing systems combine **all three** elements — feature-engineered ensemble learning, real-time performance, and built-in explainability — into a single, deployable, full-stack application. This project aims to bridge that gap.

5. Objectives

The primary objectives of this project are as follows:

1. To design and implement a robust URL feature extraction module capable of deriving 30+ lexical, domain-based, and content-based features indicative of phishing behavior.
 2. To train and rigorously evaluate multiple individual machine learning classifiers (Random Forest, XGBoost, Support Vector Machine, Logistic Regression, LightGBM, and CatBoost) on a balanced dataset of phishing and legitimate URLs.
 3. To construct a soft-voting ensemble classifier that combines the strengths of individual models to achieve a target detection accuracy of 95% or higher.
 4. To integrate SHAP-based explainability into the prediction pipeline, enabling the system to identify and communicate the specific features that most influenced each classification decision.
 5. To develop a RESTful API (using Django REST Framework) that exposes the trained model for real-time URL classification, supporting both single and batch URL submissions.
 6. To design and implement a responsive, user-friendly web interface (using React) that allows end users to submit URLs, view classification results, and understand the reasoning behind each prediction through visual explanations.
 7. To evaluate the complete system's performance in terms of accuracy, precision, recall, F1-score, and response latency, and to document findings comprehensively.
-

6. Scope of the Project

6.1 In Scope

- Extraction of URL-based features (lexical, host-based, and content-based) without requiring third-party paid APIs
- Training and evaluation of multiple supervised ML classifiers
- Construction of an ensemble model for improved accuracy
- Implementation of SHAP-based model explainability

- Development of a REST API for real-time and batch URL checking
- Development of a web-based frontend interface
- Storage of prediction history and system metrics in MongoDB
- Comprehensive testing (unit, integration, and performance testing)
- Complete technical documentation

6.2 Out of Scope

- Browser extension development (proposed as future work)
- Direct email content scanning and analysis
- Mobile native application development
- Deep packet inspection or network-level traffic analysis
- Integration with third-party paid threat intelligence services
- Real-time crawling of webpage content beyond basic HTML parsing

7. Proposed System

PhishGuard is proposed as a full-stack, ML-powered web application comprising three primary layers:

7.1 Presentation Layer A React-based single-page application that provides an intuitive interface for users to input URLs, view real-time classification results, and inspect detailed explanations of why a given URL was flagged as phishing or legitimate.

7.2 Application/Logic Layer A Django REST Framework backend responsible for:

- Extracting features from submitted URLs
- Routing extracted features through the trained ensemble model
- Generating SHAP-based explanations for each prediction
- Managing API requests, validation, and error handling

7.3 Data Layer A MongoDB NoSQL database used to persist:

- Submitted URLs and their extracted features
- Prediction results and associated confidence scores
- Historical model performance metrics
- System logs for auditing and debugging purposes

The system is designed with **modularity** and **scalability** in mind, allowing individual components (feature extraction, model inference, explainability, and presentation) to be independently updated, tested, or replaced without disrupting the overall pipeline.

8. System Requirements

8.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i5 (or equivalent)

Component	Minimum Requirement
RAM	8 GB
Storage	10 GB free space
Internet	Required for dataset download and testing

8.2 Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS, or Linux
Backend Language	Python 3.9+
Frontend Runtime	Node.js 18+
Database	MongoDB 8.0+
Version Control	Git & GitHub
IDE	Visual Studio Code (recommended)

9. Proposed Methodology

The development of PhishGuard follows a structured, phase-wise machine learning and software engineering pipeline:

9.1 Data Collection Phishing URL samples are sourced from PhishTank, an open, community-driven repository of verified phishing URLs updated hourly. Legitimate URL samples are curated from well-known, verified domains to construct a balanced dataset of approximately 10,000 URLs (50% phishing, 50% legitimate).

9.2 Data Preprocessing Collected URLs are cleaned to remove duplicates, malformed entries, and inconsistent labeling. The dataset is then split into training (80%) and testing (20%) subsets using stratified sampling to preserve class balance.

9.3 Feature Engineering For each URL, a custom feature extraction module derives over 30 distinct features, categorized as follows:

- **URL Structure Features** (9): including URL length, presence of '@' symbol, IP address usage, special character count, and use of HTTPS
- **Domain-Based Features** (4): including top-level domain (TLD), subdomain count, and SSL certificate presence
- **Content-Based Features** (4): including presence of forms, and internal/external link counts
- **Advanced Features** (13+): including Shannon entropy of the URL string, use of URL shortening services, presence of suspicious keywords (e.g., "verify," "secure," "confirm"), and composite phishing indicator flags

9.4 Model Training Individual classifiers are trained on the extracted feature set, including:

- Random Forest Classifier

- XGBoost (Extreme Gradient Boosting)
- Support Vector Machine (SVM)
- Logistic Regression
- LightGBM
- CatBoost

Each model is evaluated independently using k-fold cross-validation to ensure robustness and to mitigate overfitting.

9.5 Ensemble Construction The best-performing individual classifiers are combined using a **soft-voting ensemble** approach, wherein each model contributes a probability estimate for each class, and the final prediction is determined by averaging these probabilities. This approach leverages the complementary strengths of different algorithms, typically resulting in higher overall accuracy and robustness compared to any single model.

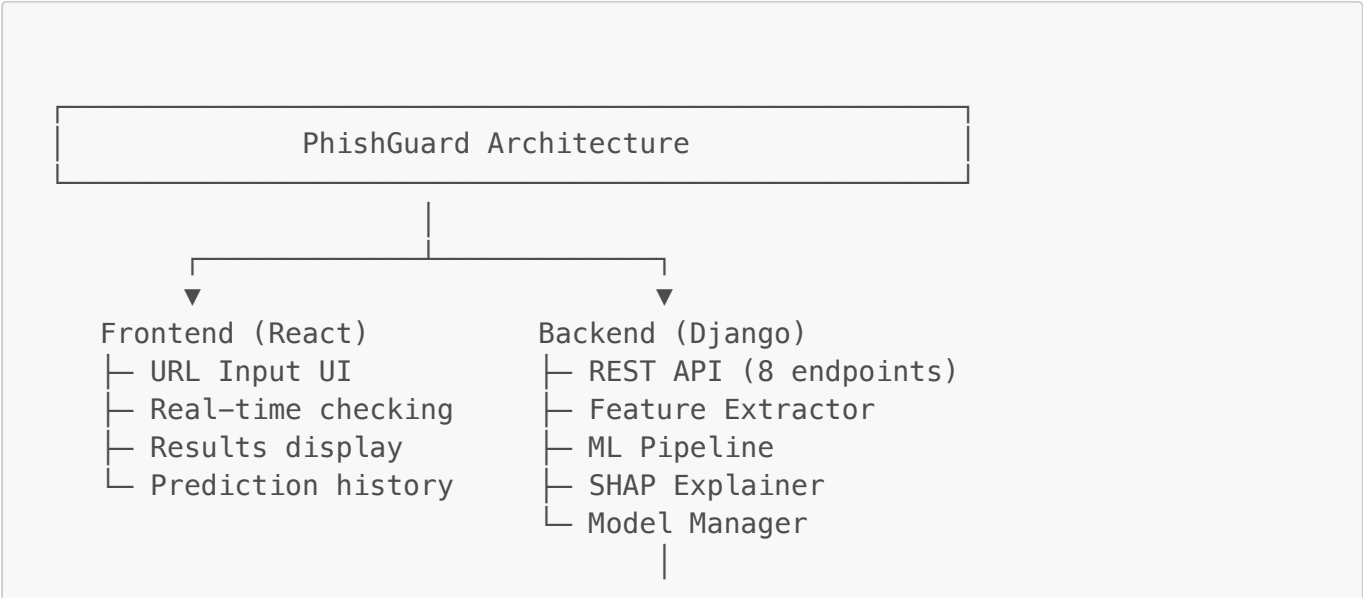
9.6 Explainability Integration SHAP (SHapley Additive exPlanations) is applied to the trained ensemble model to compute feature importance values for each individual prediction. This allows the system to generate human-readable explanations such as: *"This URL was flagged as phishing primarily due to the presence of an IP address (35% contribution), an unusually long URL (28% contribution), and the use of a URL shortening service (20% contribution)."*

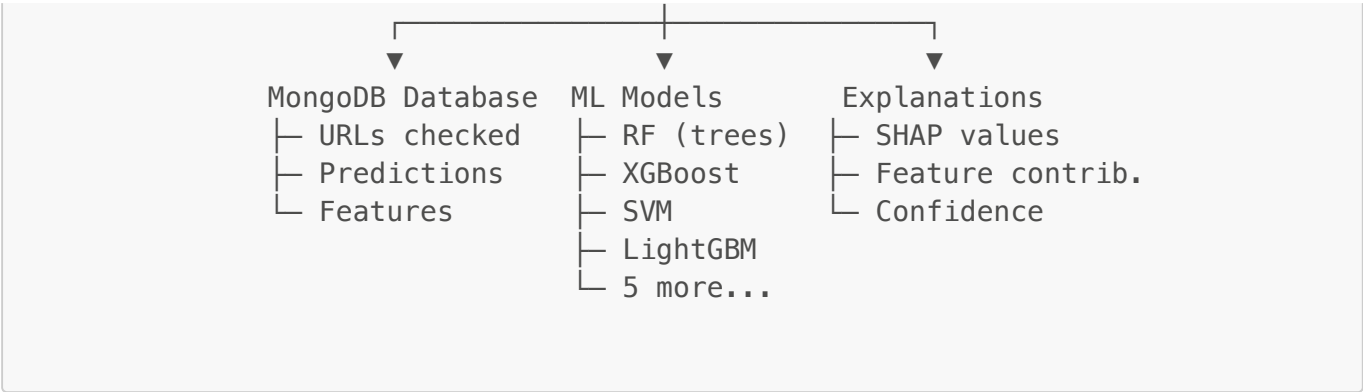
9.7 API Development The trained model, feature extractor, and SHAP explainer are wrapped within a Django REST Framework API, exposing endpoints for single URL checks, batch URL checks, prediction history retrieval, and system statistics.

9.8 Frontend Development A React-based frontend consumes the REST API, providing users with an intuitive interface to submit URLs and visualize both the classification result and the underlying explanation through interactive charts and confidence indicators.

9.9 Testing and Validation The complete system undergoes unit testing (for feature extraction and model logic), integration testing (for API endpoints), and performance testing (for response latency), targeting a minimum of 80% code coverage.

10. System Architecture





Data Flow Description:

1. A user submits a URL through the React frontend.
2. The frontend sends an HTTP POST request to the Django REST API.
3. The backend's feature extraction module parses the URL and generates a 30+ dimensional feature vector.
4. The feature vector is passed to the trained ensemble model, which returns a classification (phishing/legitimate) along with a confidence score.
5. The SHAP explainability engine computes feature contribution values for the prediction.
6. The complete result — including classification, confidence score, and explanation — is stored in MongoDB and returned to the frontend.
7. The frontend renders the result, displaying the prediction along with a visual breakdown of contributing features.

11. Feature Engineering Details

Feature engineering forms the foundation of the model's predictive capability. The following table summarizes representative features extracted for each URL:

Category	Feature	Description
URL Structure	URL Length	Total character count of the URL
URL Structure	Has '@' Symbol	Indicates domain-hiding technique
URL Structure	Has IP Address	URL uses raw IP instead of domain
URL Structure	HTTPS Token	Whether URL uses HTTPS protocol
Domain-Based	Subdomain Count	Number of subdomains present
Domain-Based	TLD	Top-level domain (.com, .ru, .xyz, etc.)
Domain-Based	SSL Certificate	Presence of valid SSL certificate
Content-Based	Form Presence	Indicates login/credential forms
Advanced	Shannon Entropy	Measures randomness in URL string
Advanced	URL Shortener Use	Detects bit.ly, tinyurl, etc.
Advanced	Suspicious Keywords	Detects "verify," "secure," etc.

These features are selected based on established patterns observed in phishing literature, where attackers frequently obscure malicious intent through URL manipulation techniques.

12. Machine Learning Models Used

Model	Type	Role in System
Random Forest	Bagging Ensemble	Base classifier, handles non-linear feature interactions
XGBoost	Gradient Boosting	Base classifier, high accuracy on structured data
Support Vector Machine	Kernel-Based	Base classifier, effective in high-dimensional space
Logistic Regression	Linear Model	Base classifier, interpretable baseline
LightGBM	Gradient Boosting	Base classifier, fast training on large datasets
CatBoost	Gradient Boosting	Base classifier, handles categorical features well
Voting Ensemble	Meta-Model	Combines all base classifiers via soft voting
SHAP	Explainability Tool	Computes feature importance for predictions

Model performance is evaluated using standard classification metrics: **Accuracy, Precision, Recall, F1-Score, and ROC-AUC**, with a target accuracy threshold of 95% or higher on the held-out test set.

13. Technology Stack

Layer	Technology Used	Justification
Frontend	React 18, TypeScript	Component-based, type-safe, widely adopted
Styling	Tailwind CSS	Rapid, consistent UI development
Backend	Python, Django, DRF	Mature framework, excellent ML library support
Database	MongoDB	Flexible schema, suitable for varied prediction data
ML Libraries	scikit-learn, XGBoost, LightGBM, CatBoost	Industry-standard, well-documented
Explainability	SHAP	Theoretically grounded, model-agnostic
Version Control	Git & GitHub	Industry-standard collaboration tool
Testing	Pytest	Robust Python testing framework

14. Expected Outcomes

Upon completion, this project is expected to deliver:

1. A fully functional, deployable web application capable of classifying URLs as phishing or legitimate with an accuracy of 95% or higher.
 2. A REST API suitable for integration into third-party applications requiring phishing detection capabilities.
 3. An explainable prediction system that builds user trust by clearly communicating the reasoning behind each classification.
 4. A comprehensive dataset and trained model artifacts that can serve as a foundation for future research or extensions.
 5. Complete technical documentation, including architecture diagrams, API specifications, and setup instructions.
 6. A response time of under 200 milliseconds per URL classification, suitable for real-time use cases.
-

15. Advantages of Proposed System

- **Proactive Detection:** Unlike blacklist-based systems, PhishGuard can identify previously unseen phishing URLs based on learned patterns rather than requiring prior manual verification.
 - **Explainability:** SHAP integration builds user trust and enables security analysts to audit and refine the system over time.
 - **High Accuracy:** Ensemble learning combines the strengths of multiple algorithms, reducing the likelihood of both false positives and false negatives.
 - **Real-Time Performance:** Optimized feature extraction and cached model inference ensure sub-200ms response times.
 - **API-First Design:** The system can be easily integrated into browsers, email clients, or other security tools via its REST API.
 - **Scalability:** The modular architecture allows for horizontal scaling and independent updates to individual system components.
-

16. Limitations

- The system's accuracy is dependent on the quality and diversity of the training dataset; highly novel attack patterns not represented in training data may reduce detection accuracy.
 - Domain age and WHOIS-based features may be limited by API rate limits or unavailability of registration data for certain domains.
 - The system currently focuses on URL-based analysis and does not perform deep visual or DOM-based webpage analysis, which could catch certain visually-deceptive phishing pages.
 - Real-time SSL certificate verification may introduce minor latency for URLs with slow-responding servers.
-

17. Future Scope

- Integration as a **browser extension** for real-time warning overlays while browsing.
- Extension to **email content scanning** for phishing detection within email clients.

- Incorporation of **deep learning models** (e.g., transformer-based URL encoders) to capture more complex patterns.
- Development of a **mobile application** for on-the-go URL verification.
- Implementation of **continuous learning**, allowing the model to be periodically retrained on newly reported phishing URLs.
- **Threat intelligence dashboard** for security teams to monitor phishing trends over time.

18. Project Timeline

Phase	Duration	Key Deliverables
Phase 1: Planning & Setup	Week 1	Repository setup, dataset collection
Phase 2: Feature Engineering	Week 2	Feature extraction module
Phase 3: Model Development	Week 2–3	Trained ensemble model, evaluation report
Phase 4: Backend Development	Week 3	REST API with prediction endpoints
Phase 5: Frontend Development	Week 3–4	Web interface with result visualization
Phase 6: Integration & Testing	Week 4	Full-stack integration, test coverage
Phase 7: Documentation & Submission	Week 4	Final report, presentation, deployment

19. Team Contribution

Team Member	Primary Responsibility
[Your Name]	Machine Learning Pipeline, Backend Architecture, Project Coordination
[Member 2]	REST API Development, Database Design
[Member 3]	Frontend Development, UI/UX Design
[Member 4]	Testing, Documentation, Deployment

All team members contributed collaboratively to feature engineering discussions, model evaluation, and final system integration.

20. Conclusion

PhishGuard represents a practical and academically grounded application of machine learning to a pressing real-world cybersecurity challenge. By combining rigorous feature engineering, ensemble learning techniques, and model explainability through SHAP, the system aims to overcome the key limitations of existing phishing detection approaches — namely, their reactive nature and lack of transparency.

The proposed full-stack implementation, encompassing a Django-based backend, MongoDB data layer, and React frontend, ensures that the system is not merely a theoretical model but a deployable, production-ready application capable of real-time use. Through this project, the team aims to

demonstrate proficiency across the full software development lifecycle — from data collection and machine learning model development to API design, frontend engineering, and system testing — while contributing a meaningful tool toward combating one of the internet's most persistent security threats.

21. References

1. PhishTank — Open Phishing Site Database. Available at: <https://www.phishtank.com>
 2. Canadian Institute for Cybersecurity, University of New Brunswick. *ISCX-URL-2016 Dataset*. Available at: <https://www.unb.ca/cic/datasets/url-2016.html>
 3. Lundberg, S. M., & Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS).
 4. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
 5. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
 6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
 7. Django Software Foundation. *Django REST Framework Documentation*. Available at: <https://www.django-rest-framework.org>
 8. MongoDB Inc. *MongoDB Documentation*. Available at: <https://docs.mongodb.com>
 9. Anti-Phishing Working Group (APWG). *Phishing Activity Trends Report*. Available at: <https://apwg.org/trendsreports/>
-

End of Synopsis